2D Embeddings for Length Generalization in Character-Based Transformers (CBT)

Neithan Vargas Vargas
neithanvarvar@gmail.com

Abstract

Character-level Transformers, which avoid subword digit merging, still fail to extrapolate multi-digit addition under standard 1D positional encodings (scoring 4.6–18.7% on 14–16 digits and collapsing to 0.0% on 18- to 20-digit sums across 1D baselines in our experiments). We show that supplying place-value coordinates through **Decoupled 2D-RoPE** removes this failure at small scale, and that the same coordinate scheme extends to multi-turn tool-calling dialogues without degrading them. In the 18.95M-parameter **Character-Based Transformer (CBT-18.95M)**, coordinates are assigned by a deterministic rule-based parser within the forward pass. Evaluated on single training runs, CBT-18.95M (`champion_final.pt`) achieves 98.9% (89/90) on in-distribution 8–15 digit sums, 100.0% (240/240) exact match on random multi-carry sums from 16 to 50 digits (digits 16–25 are seen during arithmetic replay, 30–50 are unseen), and 70.0% at 60 digits and 40.0% at 70 digits. Tool routing is 80.8% (21/26) on a phrasing suite with audited entities, with 50.0% exact argument extraction; exact copying of 18 fresh out-of-distribution entities is 55.6% (10/18). The model uses coordinate shift masking with $k = 45$ and a weight-tied recurrence ($R = 3$, adopted as an architectural choice for parameter-efficient depth rather than an addition-specific mechanism). It fits in a compact 36 MB FP16 weight footprint; CPU throughput with an unoptimized Python coordinate parser is reported in Table 3B.

1. Introduction

A foundational question in machine learning is whether autoregressive neural networks can internalize and execute exact sequential algorithms invariant to sequence length. The prevailing paradigm in contemporary language modeling scales autoregressive Transformers [1, 2] to tens or hundreds of billions of parameters over subword Byte-Pair Encoding (BPE) vocabularies. While these models exhibit remarkable fluency on conversational tasks, small Transformers trained from scratch notoriously fail on basic algorithmic tasks like multi-digit addition when tested beyond the sequence lengths seen during training [3, 4, 5, 6, 7, 8]. Frontier models with chain-of-thought or tool access do considerably better; our statements concern small models trained from scratch. While such a Transformer can memorize or interpolate addition patterns up to its training ceiling N_{train}, its accuracy drops precipitously beyond that boundary (falling to 4.6% on 14–16 digits and collapsing to 0.0% on 18- to 20-digit sums in our experiments).

1.1 Algorithmic Arithmetic and the Success of 2D Embeddings

Multi-digit integer addition serves as the canonical testbed for algorithmic length generalization: it possesses an unbounded length axis and is governed by a compact, deterministic finite-state machine (FSM) operating right-to-left from the units column ($i = 0$) to the most significant column ($i = N - 1$):

$$s_i = (a_i + b_i + c_{i-1}) \bmod 10, \quad c_i = \left\lfloor \frac{a_i + b_i + c_{i-1}}{10} \right\rfloor \tag{1}$$

where $a_i, b_i \in \{0, \ldots, 9\}$ are the single-digit operands at column i, $c_i \in \{0, 1\}$ is the binary carry bit emitted to column $i+1$, and $c_{-1} = 0$. Because the entire transition table consists of only $10 \times 10 \times 2 = 200$ discrete rules, executing addition requires $\mathcal{O}(N)$ identical state transitions regardless of whether $N = 4$, $N = 15$, or $N = 50$.

When we trained a standard 18.91M Transformer ($d_{\text{model}} = 512, L = 6, n_{\text{head}} = 8$) with standard one-dimensional rotary embeddings (1D-RoPE [9, 8]) on 1- to 12-digit additions with explicit column scratchpads [10, 11], it failed completely beyond its training horizon: achieving 98.4% on seen 1–12 digits and 4.6% on moderate 14–16 digits, before falling to **0.0% accuracy on unseen 18- to 20-digit sums** (Table 7). Diagnosing these runs revealed that 1D positional encodings assign a flat, monotonically increasing index $t = 0, \ldots, T - 1$. At unseen sequence lengths ($T \approx 715$ characters for 50 digits, scaling at ≈ 14.3 tokens per digit), relative distances $(m-n)$ between an answer column and its operand digits take values never encountered during training, inducing positional phase drift and repetitive zero loops where the model repeats blocks like `(0+0+0)=00` indefinitely:

$$P\big(\texttt{"(0+0+0)=00"} \;\big|\; \texttt{"(0+0+0)=00"}^{\times m}\big) \approx 1 \tag{2}$$

When alternative 1D architectures were evaluated under identical budgets (Looped Transformers [12, 13], Hi-

erarchical Reasoning Models, Gated Linear Recurrent Units, Harmonic Oscillators, and discrete Abacus place embeddings [7]), all nine baselines fell to 0.0% on 18–20 digits (Table 7).

To overcome this failure, we structured the problem onto a **two-dimensional (2D) coordinate grid** $(p_t, \tau_t) \in \mathbb{N}^2$, where macro row p tracks decimal place-value rank (10^p) and micro column $\tau \in [0, 9]$ tracks the intra-column execution step (Table 1A). Using **Decoupled 2D Rotary Position Embeddings (2D-RoPE)**, matching digits across operands, scratchpad, and answer share identical macro coordinates ($\Delta p = 0$). We found that **2D embeddings perform exceptionally well on arithmetic**: by preserving the decimal place-value structure of digits invariant to problem length, 2D coordinates eliminate 1D phase drift and enable reliable length extrapolation far beyond the training boundary (e.g., our 1.56M HRM-RoPE baseline extrapolates to 20 digits using Decoupled 2D-RoPE alone; Table 3).

1.2 From 2D Digits to 2D Characters: The Core Hypothesis

Standard language models rely on subword BPE tokenization, which clumps arbitrary character and digit substrings into irregular multi-character tokens:

```
"12345678" --BPE--> ["12", "3", "45", "67", "8"]
"87654321" --BPE--> ["8", "7", "65", "43", "21"]
```

A single token like `"45"` fuses the 10^3 place (4) and the 10^2 place (5) into one vector at one 1D index, breaking 1-to-1 place-value alignment between operands (Table 6).

While character-level Transformers avoid these subword merging artifacts and preserve character-level boundaries [14, 15, 16], embedding-free variants that feed fixed one-hot inputs directly into the blocks have matched or outperformed embedding-table models on translation benchmarks [17], and they still fail to extrapolate multi-digit addition under standard 1D positional encodings, succumbing to identical phase drift. Having established that Decoupled 2D-RoPE fixes addition collapse by maintaining invariant decimal place-value coordinates, we formulated our central hypothesis: *Supplying place-value coordinates through Decoupled 2D-RoPE directly to atomic characters removes this extrapolation collapse in character-level models, and this exact coordinate scheme can be extended to multi-turn tool-calling dialogues without degrading them.*

To test this, we unify arithmetic and structured dialogue into a single character representation: 1. We decompose inputs into **atomic characters** paired with 14 structural discourse tags (e.g., <USER>, <THINK>, <CALL>, <RESULT>, <ANSWER>). 2. In arithmetic mode, characters follow decimal column coordinates (p, τ) (Table 1A). In dialogue mode, characters are organized onto a **2D discourse matrix** (Table 1B), where macro row $p \in \{0..4\}$ indexes the functional discourse turn and micro column τ tracks character position within that turn.

We emphasize that discourse-mode coordinates (p = turn index) represent an architectural design choice to permit unified forward execution across arithmetic and tools—not a claim that 2D coordinates improve natural language text or tool routing over 1D. Indeed, our tokenizer ablations (Table 6) show that a standard 1D atomic-character model already scores 100% (50/50) on structured tool calling. Evaluating tool agency serves to demonstrate that Decoupled 2D-RoPE does not compromise standard character-level capabilities, allowing a single compact model to perform both tasks without mode switching.

Unlike 1D indices $t \in \{0, 1, \ldots, T-1\}$ that grow along a flat line, CBT places atomic characters onto a structured 2D coordinate grid. **Part A (Arithmetic Mode)**: Macro row p tracks the decimal column rank (10^p) so matching digits in the prompt, scratchpad, and answer share identical p ($\Delta p = 0$), while micro column $\tau \in [0, 9]$ tracks the 10-step column template. **Part B (Discourse Mode)**: Macro row $p \in \{0..4\}$ tracks the functional discourse block, while micro column τ tracks the character index within that block.

1.3 Hypothesizing the 18M CBT Architecture

To test this hypothesis and achieve maximal algorithmic and tool performance within a compact edge footprint (< 20M parameters, ˜36 MB FP16 weights), we hypothesized the unified **Character-Based Transformer (CBT-18.95M)** architecture by combining four complementary mechanisms: 1. **Atomic Character Tokenization ($V = 115$)**: Preserves exact character boundaries, eliminating subword tokenization chunking and enabling faithful 1-hop pointer copying for tool arguments. 2. **Decoupled 2D-RoPE**: Rotates half of each attention head with macro row p and the other half with micro column τ, anchoring cross-attention invariant to sequence length in both arithmetic and discourse modes. 3. **Coordinate Shift Masking ($k = 45$)**: Randomly shifts macro coordinates during training, exposing rotary frequencies to an extended coordinate horizon ($p_{\max} = N_{\text{train}} + k = 25 + 45 = 70$ for the champion's 1–25-digit arithmetic replay) to enable zero-shot extrapolation beyond the training digit range. 4. **Weight-Tied Recurrent Depth ($R = 3$)**: Unrolls $L = 6$ physical Transformer blocks across $R = 3$ recurrent passes ($L_{\text{eff}} = 18$), adopted as an architectural choice to provide deep non-linear representation within a compact 36 MB FP16 footprint without parameter inflation. As detailed in Section 1.4 and our 100k ablations (Table 8), recurrence is not claimed as an empirically necessary mech-

Table 1: How the Character-Based Transformer (CBT) organizes characters as a 2D matrix (p, τ).

Part A — Arithmetic 2D Matrix Mode : prompt `47+85=`, scratchpad `<THINK>(0+7+5)=12(1+4+8)=13</THINK>`, answer `=132` (the middle of the scratchpad is elided in the 1D index).

Region	4	7	+	8	5	=	<THINK>	(	0	+	7	+	5	)	=	1	2	</THINK>	=	1	3	2
1D index t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	28	29	30	31	32
Macro p_t (row)	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0
Micro τ_t (col)	0	0	0	0	0	0	0	0	1	2	3	4	5	6	7	8	9	0	0	0	0	0

Part B — Natural Language Discourse 2D Matrix Mode: multi-turn tool-calling dialogue.

Discourse row	<USER>	W	h	a	t...	<THINK>	I		w...	<CALL>	s	e	a	r	c...	<RESULT>	{	"...	<ANSWER>	I	t	...
Macro p_t (row)	0	0	0	0	0	1	1	1	1	2	2	2	2	2	2	3	3	3	4	4	4	4
Micro τ_t (col)	0	1	2	3	4	0	1	2	3	0	1	2	3	4	5	0	1	2	0	1	2	3

anism for addition (which extrapolates under 2D-RoPE alone at $R = 1$) or tool routing.

1.4 Scope, Limitations, and Utility on Low-Resource Devices

Why study in-model addition when a calculator tool exists? Arithmetic here is a testbed for length generalization: it has a compact, exactly checkable algorithm and an unbounded length axis. We do not argue that an in-model adder is a practical substitute for a calculator; the tool-calling results are a separate experiment.

Limitations: 1. The coordinates (p, τ) are hand-designed and assigned by a rule-based parser, not learned; the length-generalization result is therefore conditional on correct coordinate assignment (Section 3.3) and demonstrates column-aligned addition with a fixed-template scratchpad rather than general algorithmic induction. 2. Tool calling is bound to a small set of templates that overlap with training data, the OOD set contains only 18 prompts, near-exact copies are functionally failures, and we do not compare against a fine-tuned small off-the-shelf model. 3. Tool and 12d-ablation test sets are small (26 and 18 for tools, 14 for Table 8; for arithmetic we supplement our 18 structured cases with a 315-problem random sweep on CBT-18.95M in Table 3B and a 12,800-problem random sweep across four equalized 1–15d, $T = 1024$ models in Table 8B), every configuration is a single training run, and we report no seed variance. 4. In Table 7, the 18.91M 1D baselines were trained on 1–12 digits with a 512-token context window; because scratchpad traces require ≈ 14.3 characters per digit, traces for 25 digits (≈ 360 tokens) and 35 digits (≈ 500 tokens) fit within this 512-token window, where the baselines fail from positional drift rather than context clipping, whereas 50-digit traces (≈ 715 tokens) exceed 512 tokens (Table 8B addresses this with equalized 1024-token models). At 70 digits (≈ 1000 tokens) and 80 digits (≈ 1149 tokens), traces approach or exceed the 1024 context window, confounding arithmetic failure with context truncation. Several Table 7 baselines are strawmen for arithmetic (e.g., naive BPE without digit splitting), and our Abacus baseline (0.0% in-distribution) reflects an under-tuned implementation rather than the method of McLeish et al. [7]. 5. The only algorithmic task is addition; we do not test tasks where coordinates cannot be assigned by a simple parser (such as multiplication or sorting). 6. On a laptop CPU (Intel Core Ultra 5 125H, FP32), batched KV-cached inference runs at 141–394 tokens/sec (1.1–5.3 tokens/sec uncached single-stream), and we do not benchmark mobile ARM hardware or INT8 quantization. 7. Recurrent depth showed no benefit in our addition ablation (Table 8), and the 1.56M HRM-RoPE model (Table 3) also extrapolates to 20 digits using Decoupled 2D-RoPE alone, confirming that the 2D coordinate mapping—not recurrence or the multi-stage curriculum—is the primary driver of addition extrapolation.

Practical Utility on Low-Resource Devices: We make an explicit practical claim regarding scale. Scaling CBT as-is to frontier LLM sizes would not make practical sense: because CBT generates text *character-by-character* and evaluates $R = 3$ recurrent passes per character, its autoregressive inference latency is several times slower per word than subword BPE models. CBT's primary utility is in **low-resource local inference**: learning intent routing, structured JSON/API tool calling, and exact arithmetic within a tiny 100k to 18.95M parameter footprint suitable for on-device coprocessors. The 36 MB figure reflects the FP16 weight size only: with $R = 3$, per-token compute is roughly that of a ˜57M-parameter dense model and the KV cache grows with character sequence length. On a laptop CPU (Intel Core Ultra 5 125H, FP32), batched KV-cached decoding achieves 141–394 character tokens/sec across 16–100 digit traces (and 1.1–5.3 tokens/sec uncached single-stream); we have not benchmarked mobile ARM hardware or INT8 quantization.

2. Empirical Results

We first present the evaluation of the final **CBT-18.95M** checkpoint (`champion_final.pt`). It starts

from the pre-SFT base `champion_base.pt` (Stage 1 Pretraining, Stage 2 Search Copy SFT, Stage 3 All-Tools 1-Hop SFT) and is further trained with the V4 SFT curriculum (45% multi-tool dialogues, 35% search copy, 20% arithmetic replay with 1–25-digit operands, shift masking $k = 45$). Because the replay covers 1–25 digits, **16–25 digits are seen in training for champion, and zero-shot extrapolation begins at 26 digits**; the 1–15 digit framing applies to the pre-SFT base only. The prompts and entities of every champion test below were audited against the champion training corpus. Earlier champion numbers were withdrawn because their test sets overlapped that corpus. All evaluations use deterministic greedy decoding (temperature = 0.0, arg max), and each table labels digit ranges as seen or unseen. We then dissect its performance against tokenization ablations, 1D baseline comparisons, and isolated 100k-parameter component ablations.

2.1 Unified Capabilities of CBT-18.95M: Sums, Multi-Tool Routing, and OOD Copying

Tables 2, 3, 3B, 4, and 5 present the evaluation of our final **CBT-18.95M** model (`champion_final.pt`):

1. **Extrapolation Across 16–100 Digits (Tables 2, 3, and 3B)**: The champion was trained with 1–25-digit arithmetic replay and shift masking $k = 45$. On random multi-carry sums it achieves **240/240 (100.0%; CI** $[98.4, 100.0]\%$**)** from 16 to 50 digits, including 18/18 structured 16–50 digit cascades (e.g., $\underbrace{99\ldots99}_{50}+1 = 10^{50}$). Only 16–25 digits are seen in training; 30–50 digits are unseen but still exact on every problem. At 55 digits it is 20/20, then accuracy falls to **70.0% (14/20)** at 60 digits, **40.0% (6/15)** at 70 digits, and **0.0%** at 80 and 100 digits. In-distribution sums at 8, 12, and 15 digits score **89/90 (98.9%)**, with the single miss at 15 digits. A separate teacher-forced mixed-length suite (30 per length) gives 24/30 at both 60 and 70 digits, so the 60–70 digit breakdown varies between suites and should be read as an approximate region. The earlier 1–15 digit extrapolation claims were made for the baseline, not champion.
2. **Multi-Tool Phrasing (Table 4, 80.8% Routing, 50.0% Exact Args)**: On the 26-case, 9-schema suite with audited fresh entities, champion routes **21/26 (80.8%)** and copies arguments exactly in **13/26 (50.0%)**. Weather and time-zone entities drift (`Lagnes`, `Auccra`, `HST/AuctT/AtT`), a nested database query emits prose, and the fetch, event, and file tools are routed to `search`. The 90-dialogue tool benchmark passes 90/90, but its check only looks for tags and does not test argument correctness. The earlier 100% routing and 96.2% exact result is withdrawn because its prompts overlapped the champion training data.
3. **Zero-Shot OOD Copying (Table 5)**: On 18 fresh prompts, champion produced well-formed tags and routed all 18 to the intended tool. Exact copies were **10/18 (55.6%)**; eight drifted, mostly on multi-word proper nouns and rare words (`Tbilisi` to `Thilisisi`, `Yggdrasil` to `zggdrasil`). Exact copies include single words and short phrases, and a km-inversion and calculator expression. On an independent 30-case unseen benchmark (Section 4 of the clean report), champion reaches **17/30 exact (56.7%)** versus **4/30 (13.3%)** for the baseline (two-sided Fisher exact $p = 0.0009$). The copy fidelity is therefore far below the arithmetic result.

2.2 Isolated Component Ablations of CBT Mechanics (4.24M Token Budget)

Because we tested multiple combinations of CBT components that achieved similar partial extrapolation scores before converging on our final configuration, we conducted a controlled 5-model factor ablation (Table 8) using the exact 100k-parameter budget (`100,672` parameters, $L = 2, d_{\text{model}} = 64, n_{\text{head}} = 4, V = 27$) and the exact same training token volume ($4,236,029$ tokens, 2,000 steps at batch size 32) as the 100k reference model:

1. **1D-RoPE Alone or With Recurrence Fails Completely (Models 1 & 2)**: Under 1D-RoPE, neither $R = 1$ (Model 1, 21.4% overall) nor $R = 3$ recurrence (Model 2, 14.3% overall) solves even a single unseen 14–20 digit problem (0.0% extrapolation; loss stalls above `0.13`).
2. **Decoupled 2D-RoPE Is the Primary Factor (Model 1 vs. Model 3)**: The clean single-factor contrast for positional geometry is Model 1 (1D-RoPE, $R = 1, k = 0$) versus Model 3 (Decoupled 2D-RoPE, $R = 1, k = 0$): replacing 1D-RoPE with 2D-RoPE lowers training loss from `0.1864` to `0.0011` and raises accuracy to 100.0% on seen 1–12d and unseen 14–16d sums (78.6%, 11/14 overall).
3. **Recurrence Is Not Needed for Addition in This Ablation (Model 3 vs. Model 4)**: Comparing Model 3 ($R = 1$) against Model 4 ($R = 3$) isolates recurrence: both score 78.6% (11/14) and final loss is slightly worse with $R = 3$ (`0.0017` vs. `0.0011`). We therefore make no claim that recurrent depth contributes to addition at 100k scale. We did not run $R = 1$ versus $R = 3$ at 18.95M, so whether recurrence matters at larger scale or on other tasks is untested.
4. **Shift Masking and Readout Termination (Model 4 vs. Model 5)**: Comparing Model 4 ($k = 0$) and Model 5 ($k = 16$) isolates shift masking on this 14-case set; because they differ on only 1–2 test

Table 2: Executive Benchmark Summary of CBT-18.95M (`champion_final.pt`)

Scores are raw accuracies with 95% Wilson confidence intervals (no pass/fail threshold is applied). Arithmetic rows use teacher-forced greedy agreement with the gold scratchpad, equivalent to greedy reproduction of the sum; the sweep rows use free greedy decoding. OOD exact match is the headline metric over the 18 fresh prompts of Table 5.

Benchmark suite / metric	Scope / test cases	Training exposure	Final score	95% Wilson CI
1. Standalone sums (2D math)				
Tier 1: in-distribution column addition	8, 12, 15 digits ($n = 90$; 30 per length)	**Seen (1–25d replay)**	**98.9% (89/90)**	[94.0, 99.8]%
Tier 2: structured cascades and boundary cases	16–50 digits ($n = 18$, incl. $99\ldots9+1$)	Seen 16–25d; unseen 30–50d	**100.0% (18/18)**	[82.4, 100.0]%
Tier 3: random multi-carry	16, 20, 25, 30, 35, 40 digits ($n = 180$)	Seen 16–25d; unseen 30–40d	**100.0% (180/180)**	[97.9, 100.0]%
Tier 4: random multi-carry at horizon	45 and 50 digits ($n = 60$)	**Unseen** (+20 to +25d over replay)	**100.0% (60/60)**	[94.0, 100.0]%
Pooled random multi-carry (16–50 digits)	*8 lengths × 30 cases ($n = 240$)*	*Mixed (seen 16–25d, unseen 30–50d)*	***100.0% (240/240)***	*[98.4, 100.0]%*
Tier 5: beyond 50 digits (sweep)	55d ($n = 20$), 60d ($n = 20$), 70d ($n = 15$), 80d and 100d ($n = 10$ each)	Unseen; 80–100d exceed $p_{\max} = 70$	55d: **100.0% (20/20)**; 60d: **70.0% (14/20)**; 70d: **40.0% (6/15)**; 80d, 100d: **0.0% (0/10 each)**	See Table 3B
Tier 5b: beyond 50 digits (teacher-forced mix)	60d and 70d ($n = 30$ per length)	Unseen	60d: **80.0% (24/30)**; 70d: **80.0% (24/30)**	[62.7, 90.5]%
2. Multi-tool and search suite				
Multi-domain structured tool benchmark	90 dialogues, 6 categories	Seen tool schemas (generator seed 999)	**100.0% (90/90)**	[95.9, 100.0]%
Phrasing and edge-case routing (9 schemas)	26 phrasing and edge cases	Varied prepositions; audited fresh entities	**80.8% (21/26)**	[62.1, 91.5]%
Phrasing and edge-case exact arguments	26 phrasing and edge cases	1-hop pointer copy from <USER>	**50.0% (13/26)**	[32.1, 67.9]%
3. Zero-shot OOD queries				
OOD syntactic tag completion (<THINK>...<CALL>)	18 fresh prompts	Unseen prompts and entities	**100.0% (18/18)**	[82.4, 100.0]%
OOD tool selection (routing)	18 fresh prompts	Unseen entities	**100.0% (18/18)**	[82.4, 100.0]%
OOD 1-hop character copy, **exact match (headline)**	18 fresh prompts	Unseen entities	**55.6% (10/18)**	[33.7, 75.4]%
Unseen-30 head-to-head, exact match	30 unseen cases	champion vs. pre-SFT base `champion_base.pt`	**56.7% (17/30)** (pre-SFT base 13.3%, 4/30)	[39.2, 72.6]%

problems, the comparison is consistent with—but does not statistically establish—a benefit on internal scratchpad columns. At 12.8 epochs (5,000-sample pool), Models 3, 4, and 5 all score 78.6% (11/14) on strict final string match. Inspecting the raw generation tokens of Model 5 (2D-RoPE + $R = 3 + k = 16$) on the 20-digit cascading carry $99\ldots99 + 1$ reveals the following:

```
<THINK>(0+9+1)=10(1+9+0)=10 ...(20 cols)...
(1+9+0)=10(1+0+0)=01</THINK>=1[19 zeros]<EOS>
```

Inside <THINK>...</THINK>, Model 5's scratchpad was correct across all columns in the 14 test cases. The error is an *off-by-one readout termination* after </THINK>=, where the model emits 19 zeros instead of 20 before <EOS>. Because the answer-digit coordinates are derived from the scratchpad column count via $p = \max(0, C-1-i)$ (Section 3.3), the model itself must decide where the answer ends. Two plausible (untested) hypotheses for this off-by-one error are: (a) the clamp $p = \max(0, C-1-i)$ assigns $p = 0$ to both the true units digit ($i = C-1$) and any extra overflow digit ($i \geq C$), making the last answer digit and the post-answer position share the same macro coordinate $p = 0$; and (b) after shift masking moves $p > 0$ by $\Delta \in [0, 16]$ while $p = 0$ stays fixed, the $p = 1 \to p = 0$ boundary is under-trained at 12.8 epochs. When the exact same architecture is trained for 64 epochs on a 1,000-sample pool (`CBT-100k Ref`, 4.24M tokens) or scaled to **CBT-18.95M** (247.3M tokens), the readout error disappeared in our tests (14/14 for `CBT-100k Ref`; 240/240 random 16–50 digit problems with zero off-by-one errors for CBT-18.95M, Table 3B), though repeating a 1,000-sample pool 64 times raises the possibility of overfitting to training length distributions rather than purely resolving boundary attention.

Table 3: Itemized Arithmetic Length Extrapolation Benchmark (1 to 50 Digits), Separating Seen Training Lengths from Unseen Zero-Shot Extrapolation Lengths

Standard 1D-Char and 50k-BPE models (18.91M) and Dual-Loop HRM-RoPE (1.56M) were trained on **1–12 digits** with a 512-token context window; our **CBT-18.95M** pre-SFT base was trained on **1–15 digits**, and the champion additionally on **1–25-digit** arithmetic replay, all with a 1024-token context window. Part II (16–20 digits) is the clean comparison within the 512-token window: HRM-RoPE (1.56M) extrapolates to 20 digits using Decoupled 2D-RoPE without CBT's three-stage curriculum, whereas the 1D models fail. In Part III (25–50 digits), 25-digit traces (≈ 360 tokens) and 35-digit traces (≈ 500 tokens) fit within the 512-token context window of the 12d baselines, where 1D-Char fails due to positional drift; 1.56M HRM-RoPE was trained without shift masking ($k = 0$) and succeeds up to 20 digits, but fails at 25 and 35 digits because $p > 20$ exceeds its unshifted coordinate horizon; only 50-digit traces (≈ 715 tokens) exceed 512 tokens. Extending 1D-Char's frequency table to $T = 1024$ still fails due to drift; for strictly equalized 1–15 digit, $T = 1024$ training comparisons (including Ruoss et al. randomized positions), see Table 8B.

Digit Length	Training Exposure Status	Representative Test Problem	Expected Output	18.91M 1D-Char	18.91M 50k-BPE	1.56M HRM-RoPE	CBT-18.95M (Ours)
Part I: Seen Training Distribution ($1 \leq N \leq 12$–15 **Digits**)							
2 Digits	Seen in Training	`15+25=`	`40`	`40 [PASS]`	`40 [PASS]`	`40 [PASS]`	`40 [PASS]`
4 Digits	Seen in Training	`1234+5678=`	`6912`	`6912 [PASS]`	`6912 [PASS]`	`6912 [PASS]`	`6912 [PASS]`
8 Digits	Seen in Training	`12345678+87654321=`	`99999999`	`99999999 [PASS]`	`99999999 [PASS]`	`99999999 [PASS]`	`99999999 [PASS]`
8 Digits	Seen in Training (Zeros)	`10000000+10000000=`	`20000000`	`20000000 [PASS]`	`20000000 [PASS]`	`20000000 [PASS]`	`20000000 [PASS]`
12 Digits	Seen in Training	`123456789012+876543210987=`	`999999999999`	`999999999999 [PASS]`	`999999999999 [PASS]`	`999999999999 [PASS]`	`999999999999 [PASS]`
12 Digits	Seen in Training (Overflow)	`500000000000+500000000000=`	`1000000000000`	`1000000000000 [PASS]`	`1000000000000 [PASS]`	`1000000000000 [PASS]`	`1000000000000 [PASS]`
14 Digits	Unseen for 12d / Seen for 15d	`12345678901234+87654321098765=`	`99999999999999`	`Zero-Loop [FAIL]`	`Hallucinates [FAIL]`	`99999999999999 [PASS]`	`99999999999999 [PASS]`
14 Digits	Unseen for 12d / Seen for 15d	`50000000000000+50000000000=`	10^{14} (15 digits)	`Zero-Loop [FAIL]`	`Hallucinates [FAIL]`	6×10^{13} `[FAIL]`	10^{14} `[PASS]`
15 Digits	Max Seen Bound (CBT-18.95M)	`500000000000000+50000000000000=`	10^{15} (16 digits)	`Zero-Loop [FAIL]`	`Hallucinates [FAIL]`	Untested	10^{15} `[PASS]`
Part II: Unseen Moderate & Extreme Extrapolation ($16 \leq N \leq 20$ **Digits — Clean Comparison Within 512-Token Context**)							
16 Digits	**Unseen OOD (+1d / +4d)**	`1111222233334444+8888777766665555=`	$16 \times$ '9'	`Zero-Loop [FAIL]`	`Hallucinates [FAIL]`	$16 \times$ '9' `[PASS]`	$16 \times$ '9' `[PASS]`
18 Digits	**Unseen OOD (+3d / +6d)**	$10^{17} + 10^{17} =$	2×10^{17}	`Loops 000... [FAIL]`	`Invalid CoT [FAIL]`	2×10^{17} `[PASS]`	2×10^{17} `[PASS]`
20 Digits	**Unseen OOD (+5d / +8d)**	`1234...90 + 8765...09=` (20d)	$20 \times$ '9'	`Drift [FAIL]`	`Drift [FAIL]`	$20 \times$ '9' `[PASS]`	$20 \times$ '9' `[PASS]`
20 Digits	**Unseen OOD (+5d / +8d)**	$10^{19} + 10^{19} =$	2×10^{19}	`Loops 000... [FAIL]`	`Drift [FAIL]`	2×10^{19} `[PASS]`	2×10^{19} `[PASS]`
20 Digits	**Unseen OOD (+5d / +8d)**	$\underbrace{999\ldots999}_{20} + 1 =$	10^{20} (21 digits)	`Loops 000... [FAIL]`	`Drift [FAIL]`	10^{20} `[PASS]`	10^{20} `[PASS]`
Part III: Unseen Ultra-Long Horizon Extrapolation ($25 \leq N \leq 50$ **Digits**)							
25 Digits	**Unseen OOD (+10 Extra Digits)**	$\underbrace{999\ldots999}_{25} + 1 =$	10^{25} (26 digits)	`Drift [FAIL]`	`Hallucinates [FAIL]`	`Horizon [FAIL]`	10^{25} `[PASS]`
35 Digits	**Unseen OOD (+20 Extra Digits)**	$\underbrace{111\ldots111}_{35} + \underbrace{888\ldots888}_{35} =$	$35 \times$ '9'	`Drift [FAIL]`	`Hallucinates [FAIL]`	`Horizon [FAIL]`	$35 \times$ '9' `[PASS]`
50 Digits	**Unseen OOD (+35 Extra Digits)**	`123...890 + 876...109=` (50d)	$50 \times$ '9'	`Drift [FAIL at T=1024]`	`N/A (Context: 715 > 512)`	`N/A (Context: 715 > 512)`	$50 \times$ '9' `[PASS]`
50 Digits	**Unseen OOD (+35 Extra Digits)**	$\underbrace{999\ldots999}_{50} + 1 =$	10^{50} (51 digits)	`Drift [FAIL at T=1024]`	`N/A (Context: 715 > 512)`	`N/A (Context: 715 > 512)`	10^{50} `[PASS]`

Table 3B: Random Multi-Carry Extrapolation Sweep from 16 to 100 Digits on CBT-18.95M (`champion_final.pt`, $n = 315$ Problems, Seed 20261022)

Each length uses the stratified random mixture (70% uniform random digits with multi-carries, 15% dense carries with $a_i + b_i \geq 9$, 15% random cascade blocks) under deterministic greedy decoding, with KV-cached batched inference (`benchmarks/evaluate_random_16_to_100d.py`). champion was trained with shift masking $k = 45$ on 1–25-digit operands, so the maximum macro coordinate seen in training is $p_{\max} = 25 + 45 = 70$. champion is exact on all **240/240** problems from 16 to 50 digits and on 20/20 at 55 digits, then degrades: 14/20 at 60 digits, 6/15 at 70 digits, and 0/10 at 80 and 100 digits. The 60-digit failures occur before p reaches $p_{\max}$, so $p_{\max}$ does not mark a sharp failure point for champion. A separate teacher-forced suite with a different generator (Table 2, Tier 5b) gives 24/30 at both 60 and 70 digits. CPU timings were measured while other evaluations ran in parallel and are therefore only indicative.

Digit Length (d)	Max Column (p)	Coordinate Regime vs. $p_{\max} = 70$	Test Cases (n)	<THINK> Scratchpad Acc	Final Answer Exact Match	95% Wilson CI	(Errors / CPU Tok/s
16 Digits	$p \leq 16$	Seen in replay (1–25d)	30	**100.0% (30/30)**	**100.0% (30/30)**	[88.6, 100.0]%	0 / 245.6 tok/s
20 Digits	$p \leq 20$	Seen in replay (1–25d)	30	**100.0% (30/30)**	**100.0% (30/30)**	[88.6, 100.0]%	0 / 215.6 tok/s
25 Digits	$p \leq 25$	Seen in replay (1–25d)	30	**100.0% (30/30)**	**100.0% (30/30)**	[88.6, 100.0]%	0 / 187.4 tok/s
30 Digits	$p \leq 30$	Unseen, within $p_{\max} = 70$	30	**100.0% (30/30)**	**100.0% (30/30)**	[88.6, 100.0]%	0 / 154.7 tok/s
35 Digits	$p \leq 35$	Unseen, within $p_{\max} = 70$	30	**100.0% (30/30)**	**100.0% (30/30)**	[88.6, 100.0]%	0 / 148.3 tok/s
40 Digits	$p \leq 40$	Unseen, within $p_{\max} = 70$	30	**100.0% (30/30)**	**100.0% (30/30)**	[88.6, 100.0]%	0 / 103.6 tok/s
45 Digits	$p \leq 45$	Unseen, within $p_{\max} = 70$	30	**100.0% (30/30)**	**100.0% (30/30)**	[88.6, 100.0]%	0 / 99.0 tok/s
50 Digits	$p \leq 50$	Unseen, within $p_{\max} = 70$	30	**100.0% (30/30)**	**100.0% (30/30)**	[88.6, 100.0]%	0 / 120.7 tok/s
55 Digits	$p \leq 55$	Unseen, within $p_{\max} = 70$	20	**100.0% (20/20)**	**100.0% (20/20)**	[83.9, 100.0]%	0 / 108.8 tok/s
60 Digits	$p \leq 60$	Unseen, within $p_{\max} = 70$	20	**70.0% (14/20)**	**70.0% (14/20)**	[48.1, 85.5]%	1 / 106.4 tok/s
70 Digits	$p \leq 70$	Unseen, at $p_{\max} = 70$	15	**40.0% (6/15)**	**40.0% (6/15)**	[19.8, 64.3]%	0 / 85.9 tok/s
80 Digits	$p \leq 80$	Exceeds $p_{\max} = 70$	10	**0.0% (0/10)**	**0.0% (0/10)**	[0.0, 27.8]%	5 / 74.5 tok/s
100 Digits	$p \leq 100$	Exceeds $p_{\max} = 70$	10	**0.0% (0/10)**	**0.0% (0/10)**	[0.0, 27.8]%	0 / 71.3 tok/s
Pooled 16–40d	$p \leq 40$	**Within Seen/Unseen Range ($p \leq 40$)**	**180**	**100.0% (180/180)**	**100.0% (180/180)**	**[97.9, 100.0]%**	**0 Errors**
Pooled 16–50d	$p \leq 50$	**Full Target Range (16–50d)**	**240**	**100.0% (240/240)**	**100.0% (240/240)**	**[98.4, 100.0]%**	**0 Errors**

2.3 Strictly Equalized 1–15 Digit, $T = 1024$ Comparison with Ruoss et al. Randomized Positions (12,800 Random Problems)

Because the 18.91M 1D baselines in Table 7 were trained on 1–12 digits with a 512-token context window and without position-offset augmentation, we conducted a strictly equalized experiment (`experiments/run_equalized_15d_1024_and_large_sweep.py`, Table 8B) where all models share the exact same 100k character-level architecture ($L = 2, d_{\text{model}} = 64, n_{\text{head}} = 4, R = 1, V = 27$), the exact same 6,000-sample **1–15 digit** training set, a **1024-token context window**, and 3,000 training steps at batch size 32, evaluated on **200 stratified random multi-carry problems at every length** $(8, 12, 15, 16, 20, 25, 30, 35, 40, 45, 50, 55, 60, 70, 80, 100$

Table 4: Multi-Tool Phrasing & Edge-Case Suite on CBT-18.95M (`champion_final.pt`, 26 Audited Cases, 9 Schemas)

Routing (correct tool) is **21/26 (80.8%, CI [62.1, 91.5]%)**; exact argument extraction is **13/26 (50.0%, CI [32.1, 67.9]%)**. All 26 prompts and entities were audited against the champion training corpus. The earlier Stage 2 vs. Stage 3 comparison (100% routing, 96.2% exact) is withdrawn: its prompts overlapped the champion training data, so it does not measure generalization.

Tool Category	Cases	Correct Routing	Exact Arguments	Observed Failures
Weather (`get_weather`)	7	7	4	City drift: `Lagnes` (Lagos), `Auccra` (Accra), `Rerevan` (Yerevan)
Time (`get_time`)	3	3	0	Zone-code drift: `HST` (NZST), `AuctT` (SAST), `AtT` (WAT)
Unit (`unit_convert`)	2	2	2	None
Search (`search`)	5	5	4	`Joannes Kepler`
Calculator (`calculator`)	3	3	2	`37 * 13` for `37 * 113` (dropped digit)
Nested DB query (`database_query`)	1	0	0	Emitted prose and no `<CALL>`
Fetch API (`fetch_api`)	1	0	0	Misrouted to `search`
Create event (`create_event`)	1	0	0	Misrouted to `search`
File manager (`file_manager`)	1	0	0	Misrouted to `search`
Direct chat (no tool)	2	1	1	`Hello, how are you today?` routed to `get_weather(city="Toro")`
Total	**26**	**21 (80.8%)**	**13 (50.0%)**	

Table 5: Zero-Shot Out-of-Distribution (OOD) Generalization of CBT-18.95M (`champion_final.pt`) on 18 Fresh Prompts

All 18 prompts produced well-formed tags and were routed to the intended tool (18/18, 100.0%, CI [82.4, 100.0]%). Exact argument copies: **10/18 (55.6%, CI [33.7, 75.4]%)**. The eight drifted copies are listed below. Near-exact copies are tool-call failures (e.g., `search(query="zggdrasil")` searches the wrong string).

Result	Prompts (Entity or Argument)	Count
`[EXACT]`	bioluminescence, aerogel, stagflation, ceviche, quasars, km-to-miles inversion, calculator `61 * 27`, convolutional networks, harpsichord, art nouveau	10
`[DRIFT]`	Tbilisi → `Thilisisi`; WAT → `ATC`; nihilism → `niiilism`; Yggdrasil → `zggdrasil`; Hatshepsut → `watshepsut`; Santo Domingo → `Santo`; Ho Chi Minh City → `Cith Cith`; `crispr cas12` → `crispr cos1212`	8

digits; 3,200 problems per model, 12,800 problems total): 1. **Equalized 1D Baselines Suffer In-Distribution Optimization Failure**: In this 100k, 1024-context setting, standard 1D-RoPE (`loss=0.0982`), 1D-RoPE with translation offsets $\Delta \sim \mathcal{U}\{0..512\}$ (`loss=0.0958`), and Ruoss et al. randomized positions (`loss=0.2318`) failed to converge in-distribution, scoring **0.0% (0/600; 95% Wilson CI** $[0.0, 0.6]\%$**)** on seen 8-, 12-, and 15-digit sums. Under flat 1D coordinates in a 1024-token context, 100k parameters and 3,000 steps are insufficient for the attention heads to learn the variable-distance routing required for column scratchpads. This test shows that 1D models fail to optimize the task under equalized conditions, rather than isolating length extrapolation per se; the cleaner evidence for extrapolation failure after successful in-distribution learning is provided by Table 7, where 18.91M 1D-Char achieves 98.4% in-distribution but collapses to 4.6% at 14–16d and 0.0% at 18–20d. 2. **Why Ruoss et al. [18] Randomized 1D Positions Do Not Substitute for 2D Column Alignment**: Sampling sorted random 1D positions $0 \leq t_1 < t_2 < \cdots < t_T < 1024$ during training causes training loss to stall at `0.2318` and yields **0.0% (0/200)** across all tested lengths. In a flat 1D sequence without explicit index hints, the model relies on consistent relative token distances to locate corresponding operand digits; randomizing inter-token 1D gaps destroys those relative offsets, whereas shifting the macro coordinate $p \leftarrow p + \Delta$ in Decoupled 2D-RoPE

Table 6: The Tokenization and Positional Geometry Matrix

Comparing our 2D Matrix Character + Control-Token CBT formulation against 1D Character models, Subword BPE models (4k and 50k vocabularies), Digit-Split 4k BPE, and Learned 2D positional tables, explicitly contrasting **Seen Training Digits** vs. **Unseen Extrapolation Digits**. Vocabulary sizes differ by variant: 115 is the full CBT-18.95M tokenizer, 111 the CBT-200k and Learned-2D variants, and 27 the digit-and-symbol character models. All rows showing `/50` in the Tool Calling column were evaluated on the identical 50-dialogue structured tool suite (6 domains), whereas CBT-18.95M was evaluated on the 26-case 9-schema 1-hop suite of Table 4. Rows were trained with different digit ranges and data and are not a controlled 2×2 study. The unseen-sum column pools 14–20 digits, whereas Table 7 reports 14–16 digits separately (hence 2.3% here and 4.6% there for the 1D character model). The Structured Cascade column reports the hardest single-carry chain ($99\ldots9+1$) at unseen 20 digits for the 12d models and 50 digits for CBT-18.95M (where CBT-18.95M scores 100.0% on the structured cascade case, contrasting with 100.0% on random 50-digit multi-carries in Table 3B).

Tokenizer Granularity	Positional Geometry	Vocab (V)	Trained Digits	Tool Calling	Seen Sums (In-Dist)	Unseen Sums (14–20d)	Structured Cascade
Subword BPE (50k)	1D-RoPE (Flat Line)	50,304	1–12 Digits	N/A (Word-CoT)	99.1% (1–12d)	**0.0%** (Unseen 18–20d)	**0.0%** (Unseen 20d)
Subword BPE (Naive 4k)	1D / 2D-RoPE	4,000	1–12 Digits	**100.0% (50/50)**	**0.0%** (1–12d)	**0.0%** (Unseen 14–20d)	**0.0%** (Unseen 20d)
Digit-Split BPE (4k)	2D-RoPE Matrix	4,114	1–12 Digits	**100.0% (50/50)**	83.3% (1–12d)	64.3% (Unseen 14–20d)	**0.0%** (Unseen 20d)
Atomic Character	1D-RoPE (Flat Line)	27 / 115	1–12 Digits	**100.0% (50/50)**	98.4% (1–12d)	2.3% (Unseen 14–20d)	**0.0%** (Unseen 20d)
Atomic Character	NoPE + Abacus Place	27	1–12 Digits	N/A	**0.0%** (1–12d)	**0.0%** (Unseen 14–20d)	**0.0%** (Unseen 20d)
Atomic Character	Learned 2D Lookup Table	111	1–12 Digits	98.0% (49/50)	100.0% (1–12d)	**0.0%** (Capped $p > 12$)	**0.0%** (Unseen 20d)
CBT-200k (Char + Control)	**2D-RoPE Matrix** ($R = 3$)	**111**	**1–12 Digits**	**100.0% (50/50)**	**100.0% (1–12d)**	**100.0% (Unseen 14–20d)**	**100.0% (Unseen 20d)**
CBT-18.95M (Char + Control)	**2D-RoPE Matrix** ($R = 3$)	**115**	**1–25 Digits (replay)**	**80.8% (21/26)**	**98.9% (89/90, 8–15d)**	**100.0% (240/240 Rand 16–50d)**	**100.0% (18/18 Struct)**

preserves $\Delta p = p_t - p_j = 0$ between every scratchpad column and its matching operand digits. 3. **Equalized Decoupled 2D-RoPE ($k = 32$) Extrapolates Across 1,200 Unseen 16–40d Problems**: Under the exact same 100k $R = 1$ architecture and 3,000-step budget, Decoupled 2D-RoPE drops training loss to `0.0060`, achieves 69.5% (417/600) on seen 8–15 digits (86.0% at 15 digits), and achieves **81.4% (977/1,200; 95% Wilson CI** $[79.1, 83.5]\%$**)** pooled across unseen 16–40 digit random multi-carry problems—including **92.0% (184/200)** at 16d and 25d, **90.0% (180/200)** at 20d, and **85.0% (170/200)** at 30d—before dropping at 45d (**30.0%, 60/200**) and reaching 0.0% at $d \geq 50$ once column indices exceed the $p_{\max} = 15 + 32 = 47$ shift-masking support. The lower accuracy of equalized 2D (81.4% vs. 100% in CBT-18.95M) reflects partial undertraining at 3,000 steps on 100k parameters.

2.4 Ablation Analysis: Coordinate Horizon Shifting, Decoupling, and Optimization

To isolate the mechanisms governing length generalization, we conducted three targeted empirical ablations (`experiments/run_reviewer_priority_ablations.py`, Table 8C): 1. **Causal Shift Sweep ($k \in \{0, 16, 32, 64\}$)**: By varying the training position-shift offset k, we observe that the accuracy cliff moves strictly with $p_{\max} = N_{\text{train}} + k$. When $k = 0$ ($p_{\max} = 15$), accuracy drops precipitously immediately past the training frontier (77.5% at 16d, 17.0% at 20d, 0.0% at 25d). When $k = 16$ ($p_{\max} = 31$), accuracy exceeds 98.5% through 30d before falling to 60.5% at 35d and 0.0% at 40d. When $k = 64$ ($p_{\max} = 79$), the model maintains substantial accuracy well beyond 40d (69.0% at 40d, 60.0% at 45d, 41.0% at 50d, and 20.0% at 55d). This confirms that the bounded extrapolation horizon is not a failure of attention routing, but a direct consequence of the coordinate support envelope encountered during training. 2. **Place-Only Ablation (p-Only, $\tau = 0$)**: To test whether the intra-column micro-step coordinate τ is necessary or if place alignment alone suffices, we set $\tau = 0$ during both training and inference ($k = 32$). The p-only model achieves 99.8% on seen 8–15 digits and averages 96.2% across 16–30 digits (100.0% at 16d, 99.5% at 20d, 99.0% at 25d, 97.5% at 30d, 85.0% at 35d). This demonstrates that the macro place coordinate p alone carries the underlying length-invariant algorithmic scaffold. However, full 2D-RoPE provides greater robustness near the cliff boundary (58.0% vs. 41.5% at 40d; 30.5% vs. 9.5% at 45d), showing that τ provides secondary stabilization against phase dispersion during token gen-

Table 7: Cross-Architecture Comparison on Seen vs. Unseen Digit Lengths
Nine 1D/NoPE baselines (the default character model plus eight alternatives, trained on 1–12 digits; several achieve 4.6–18.7% on moderate 14–16 digit extrapolation, while all nine fall to 0.0% on unseen 18–20 digit problems) compared against the 2D-RoPE Character-Based Transformer (CBT) family. CBT-18.95M trained on 1–15 digits and the baselines on 1–12 digits; baselines used no position-offset augmentation and no tuned index hints, so this is not an equalized comparison. The MicroMath/Abacus row (0.0% even in-distribution) likely reflects an under-tuned implementation rather than a property of Abacus embeddings [7].

Architecture Paradigm	Non-Emb Params	Positional Encoding	Trained Digits	Seen In-Dist	Unseen Mod OOD	Unseen Ext OOD	Unseen Extrapolation Summary
Standard Char Transformer	18.91M	1D-RoPE	1–12 Digits	98.4%	4.6% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Subword BPE Word-CoT	18.91M	1D-RoPE	1–12 Digits	99.1%	8.0% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Philo-Math Co-Trained GPT	18.91M	1D-RoPE	1–12 Digits	94.2%	5.3% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Looped Recurrent GPT ($K = 6$)	18.91M	1D-RoPE	1–12 Digits	100.0%	12.0% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Hierarchical HRM-1D	18.91M	1D-RoPE	1–12 Digits	100.0%	18.7% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Hybrid GLRU-SSM Transformer	18.91M	1D-RoPE + Linear State	1–12 Digits	100.0%	10.7% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Harmonic ALU (H-NRM)	18.91M	Poly-Freq Oscillators	1–12 Digits	97.8%	8.0% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
MicroMath (Abacus NoPE, 1.36M)	1.36M	Discrete Place Only (p)	1–12 Digits	0.0%	0.0% (14–16d)	0.0% (18–20d)	0 Digits (Under-tuned / Collapse)
Foundation HRM-Text (219k tok/s)	1.44M	1D-RoPE + Additive Loop	1–12 Digits	66.7%	0.0% (14–16d)	0.0% (18–20d)	No length beyond 8d reliably
HRM-RoPE (Dual-Loop)	1.56M	**Decoupled 2D-RoPE**	**1–12 Digits**	100.0%	66.7% (14–16d)	**100.0% (18–20d)**	**20 Digits (+8 Unseen Digits)**
CBT-100k ($L = 2, R = 3$)	98.9k	**Decoupled 2D-RoPE**	**1–12 Digits**	**100.0%**	**100.0% (14–16d)**	**100.0% (18–20d)**	**20 Digits (+8 Unseen Digits)**
CBT-200k ($L = 4, R = 3$)	197.8k	**Decoupled 2D-RoPE**	**1–12 Digits**	**100.0%**	**100.0% (14–16d)**	**100.0% (18–20d)**	**20 Digits (+8 Unseen) + Tools**
CBT-18.95M ($L = 6, R = 3$)	**18.89M**	**Decoupled 2D-RoPE**	**1–25 Digits (replay)**	**98.9% (89/90, 8–15d)**	**100.0% (16–40d Rand)**	**100.0% (45–50d); 100.0% (55d)**	**100.0% (16–50d); 70.0% (60d), 40.0% (70d)**

eration. 3. **1D Baseline Retraining with Extended Budget**: To determine whether 1D failures in Table 8B were merely artifacts of insufficient training steps, we retrained 1D-RoPE with double the step budget (6,000 steps) and a tuned learning rate (LR $= 8 \times 10^{-4}$). The retrained model attained only 28.0% accuracy on 8-digit additions and collapsed to 0.0% on 12- and 15-digit sums (as well as all unseen lengths). In contrast to 2D models which converge smoothly to zero loss, flat 1D coordinates in long-context scratchpads create an intractable optimization landscape for compact Transformers.

Table 8: Isolated Component Ablation Study of CBT Mechanics at 100k Scale (100,672 Total Params, Identical 4.24M Token Budget)

All 6 models share the exact same 100k character-level backbone ($L = 2, d_{\text{model}} = 64, n_{\text{head}} = 4, V = 27$) and vary three factors shown in the top configuration rows: **Positional Encoding** (`1D-RoPE` vs. `Decoupled 2D-RoPE`), **Recurrent Depth** ($R = 1$ single pass vs. $R = 3$ weight-tied passes), and **Coordinate Shift Masking** ($k = 0$ off vs. $k = 16$ random column shift). Models 1–5 are trained for 2,000 steps on a 5,000-sample pool (≈ 12.8 epochs); the 6th column (`CBT-100k Ref`) trains the Model 5 configuration for 64 epochs on a 1,000-sample pool (`dense_100k_2d_rec3_full.pt`). In Model 5, the internal <THINK> scratchpad columns were exact in all 14 cases, with the 3 readout misses attributable to an off-by-1 trailing zero before <EOS>. Because the 64-epoch reference repeats a 1,000-sample pool 64 times, we cannot rule out memorization or overfitting to boundary patterns on that checkpoint. The 100k ablation uses $k = 16$, whereas CBT-18.95M uses $k = 32$ (Eq. 6). With 14 test cases (where 1–3 cases separate models), Models 3, 4, and 5 cannot be statistically distinguished, and each configuration was trained once.

Benchmark Tier / Configuration	Training Exposure	Model 1 (1D Baseline)	Model 2 (1D + Recurrence)	Model 3 (2D-RoPE Only)	Model 4 (2D + Recurrence)	Model 5 (2D + Rec + Shift)	CBT-100k Ref (2D + Rec + Shift, 64 Ep)
Positional Geometry	—	1D-RoPE (Flat)	1D-RoPE (Flat)	**Decoupled 2D-RoPE**	**Decoupled 2D-RoPE**	**Decoupled 2D-RoPE**	**Decoupled 2D-RoPE**
Recurrent Passes (R)	—	$R = 1$ (2 Layers)	$R = 3$ (6 Eff. Layers)	$R = 1$ (2 Layers)	$R = 3$ (6 Eff. Layers)	$R = 3$ (6 Eff. Layers)	$R = 3$ (6 Eff. Layers)
Random Shift Masking (k)	—	Off ($k = 0$)	Off ($k = 0$)	Off ($k = 0$)	Off ($k = 0$)	**On ($k = 16$)**	**On ($k = 16$)**
Training Pool / Epochs	4.24M Tokens	5k Pool (12.8 Ep)	5k Pool (12.8 Ep)	5k Pool (12.8 Ep)	5k Pool (12.8 Ep)	5k Pool (12.8 Ep)	1k Pool (64.0 Ep)
Final Pretraining Loss	4.24M Tokens	`0.1864`	`0.1353`	`0.0011`	`0.0017`	`0.0017`	`0.0016`
Seen In-Dist (1–12 Digits)	**Seen (1–12d)**	50.0% (3/6)	33.3% (2/6)	**100.0% (6/6)**	**100.0% (6/6)**	**100.0% (6/6)**	**100.0% (6/6)**
Unseen Mod Extrap (14–16d)	**Unseen (+2 to +4d)**	0.0% (0/3)	0.0% (0/3)	**100.0% (3/3)**	**100.0% (3/3)**	**100.0% (3/3)**	**100.0% (3/3)**
Unseen Ext Extrap (18–20d)	**Unseen (+6 to +8d)**	0.0% (0/3)	0.0% (0/3)	33.3% (1/3)	33.3% (1/3)	**66.7% (2/3)**	**100.0% (3/3)**
$10^{17} + 10^{17} =$ (18d Zeros)	Unseen (+6d)	`[FAIL]`	`[FAIL]`	`[FAIL]`	`[FAIL]`	`2e17 [PASS]`	`2e17 [PASS]`
`1234...90+ 8765...09=` (20d)	Unseen (+8d)	`[FAIL]`	`[FAIL]`	`99...99 [PASS]`	`99...99 [PASS]`	`99...99 [PASS]`	`99...99 [PASS]`
`5555...55+ 4444...45=` (20d)	Unseen (+8d)	`[FAIL]`	`[FAIL]`	`[FAIL]`	`[FAIL]`	*100% Think / Off-by-1 0*	`1e20 [PASS]`
Unseen 20d Zeros & Carry	**Unseen (+8d)**	0.0% (0/2)	0.0% (0/2)	50.0% (1/2)	50.0% (1/2)	*100% Think / Off-by-1 0*	**100.0% (2/2)**
$10^{19} + 10^{19} =$ (20d Zeros)	Unseen (+8d)	`[FAIL]`	`[FAIL]`	`2e19 [PASS]`	`2e19 [PASS]`	*19/19 Cols OK, 18 0s out*	`2e19 [PASS]`
$99\ldots99 + 1 =$ (20d Carry)	Unseen (+8d)	`[FAIL]`	`[FAIL]`	`[FAIL]`	`[FAIL]`	*21/21 Cols OK, 19 0s out*	`1e20 [PASS]`
Internal <THINK> Math Acc	All 14 Cases	21.4% (3/14)	14.3% (2/14)	78.6% (11/14)	78.6% (11/14)	**100.0% (14/14 Exact)**	**100.0% (14/14)**
Strict Final String Match	All 14 Cases	21.4% (3/14)	14.3% (2/14)	78.6% (11/14)	78.6% (11/14)	78.6% (11/14)	**100.0% (14/14)**

3. Architecture, 2D Matrix Embeddings, Tokenizer, and Training Data

In this section, we provide a complete technical specification of **CBT-18.95M**, detailing its hybrid character-control tokenizer, Decoupled 2D-RoPE embeddings, weight-tied recurrent depth, and the exact composition

Table 8B: Strictly Equalized 1–15 Digit, $T = 1024$ Controlled Comparison Including Ruoss et al. (2023) Randomized Position Encodings

All four 100k models share the exact same character-level backbone ($L = 2, d_{\text{model}} = 64, n_{\text{head}} = 4, R = 1, V = 27$), the exact same 6,000-equation **1–15 digit** training dataset, a **1024-token context window**, and 3,000 training steps at batch size 32 (evaluated on 200 random multi-carry problems per length; 3,200 problems/model, 12,800 total). In this long-context 100k setup, all three 1D variants (standard 1D-RoPE, translation offset $\Delta \sim \mathcal{U}\{0..512\}$, and Ruoss et al. randomized positions) suffered an **in-distribution optimization failure** (scoring 0/600 on seen 8, 12, and 15 digits, loss ≈ 0.10–0.23), failing to learn the task under flat coordinates. Thus Table 8B reflects an equalized training/optimization comparison; the isolated extrapolation failure after successful training is shown in Table 7 (where 18.91M 1D-Char achieves 98.4% in-distribution but drops to 4.6% at 14–16d and 0.0% at 18–20d). Equalized Decoupled 2D-RoPE ($k = 32, p_{\max} = 47$) achieves 69.5% in-distribution and reaches **81.4% (977/1,200)** across unseen 16–40 digits (reflecting partial undertraining at 3,000 steps relative to the 18.95M model's 100% score) before dropping to 0.0% beyond $p_{\max} = 47$ ($d \geq 50$). Right column shows the converged CBT-18.95M reference ($n = 30$ per length from Table 3B).

Evaluation Digit Length	Exposure Status	1D-RoPE Standard (100k, $n = 200$)	1D-RoPE + Offset (100k, $n = 200$)	1D-RoPE + Ruoss et al. (100k, $n = 200$)	Decoupled 2D-RoPE + Shift (100k, $n = 200$)	CBT-18.95M Ref ($n = 30$/len, Table 3B)
Final Training Loss (3,000 Steps)	1–15 Digits	0.09822	0.09582	0.23179	0.00597	0.00009
Seen 8, 12, 15 Digits ($n = 600$ / $n = 90$)	Seen (1–15d)	0.0% (0/600) [0.0, 0.6]%	0.0% (0/600) [0.0, 0.6]%	0.0% (0/600) [0.0, 0.6]%	**69.5% (417/600)** [65.7, 73.1]%	**100.0%** (Seen 1–15d)
of which 15 Digits *($n = 200$ / $n = 30$)*	Max Seen (15d)	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**86.0% (172/200)** [80.5, 90.1]%	**100.0%**
Unseen 16 Digits ($n = 200$ / $n = 30$)	**Unseen (+1d)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**92.0% (184/200)** [87.4, 95.0]%	**100.0% (30/30)** [88.6, 100]%
Unseen 20 Digits ($n = 200$ / $n = 30$)	**Unseen (+5d)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**90.0% (180/200)** [85.1, 93.4]%	**100.0% (30/30)** [88.6, 100]%
Unseen 25 Digits ($n = 200$ / $n = 30$)	**Unseen (+10d)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**92.0% (184/200)** [87.4, 95.0]%	**100.0% (30/30)** [88.6, 100]%
Unseen 30 Digits ($n = 200$ / $n = 30$)	**Unseen (+15d)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**85.0% (170/200)** [79.4, 89.3]%	**100.0% (30/30)** [88.6, 100]%
Unseen 35 Digits ($n = 200$ / $n = 30$)	**Unseen (+20d)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**78.0% (156/200)** [71.8, 83.2]%	**100.0% (30/30)** [88.6, 100]%
Unseen 40 Digits ($n = 200$ / $n = 30$)	**Unseen (+25d)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**51.5% (103/200)** [44.6, 58.3]%	**100.0% (30/30)** [88.6, 100]%
Unseen 45 Digits ($n = 200$ / $n = 30$)	**Unseen (+30d)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**30.0% (60/200)** [24.1, 36.7]%	**93.3% (28/30)** [78.7, 98.2]%
Unseen 50 Digits ($n = 200$ / $n = 30$)	**Boundary ($p \approx 47$)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**96.7% (29/30)** [83.3, 99.4]%
Unseen 55 Digits ($n = 200$ / $n = 20$)	**Beyond $p_{\max}$ (+8d)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	**100.0% (20/20)** [83.9, 100.0]%
Unseen 60 Digits ($n = 200$ / $n = 20$)	**Beyond $p_{\max}$ (+13d)**	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	0.0% (0/200) [0.0, 1.9]%	45.0% (9/20) [25.8, 65.8]%
Unseen 70–100 Digits ($n = 600$ / $n = 45$)	**Beyond $p_{\max}$ & Context**	0.0% (0/600) [0.0, 0.6]%	0.0% (0/600) [0.0, 0.6]%	0.0% (0/600) [0.0, 0.6]%	0.0% (0/600) [0.0, 0.6]%	2.2% (1/45; 6.7% 70d, 0% 80–100)
Pooled Unseen 16–40d ($n = 1,200$ / $n = 180$)	**Within $p_{\max} = 47$**	0.0% (0/1200) [0.0, 0.3]%	0.0% (0/1200) [0.0, 0.3]%	0.0% (0/1200) [0.0, 0.3]%	**81.4% (977/1200)** [79.1, 83.5]%	**100.0% (180/180)** [97.9, 100]%

Table 8C: Empirical Verification of Coordinate Horizon Shifting ($k \in \{0, 16, 32, 64\}$), Place-Only Ablation (p-Only), and Retrained 1D Baseline

Evaluated on $n = 200$ stratified random multi-carry problems per length (3,000 problems per model, 18,000 total across six conditions). All models share the equalized 100k architecture trained on 1–15 digits in a 1024-token context. The 2D-RoPE ($k = 32$) baseline evaluates the identical checkpoint from Table 8B on an independent random evaluation draw of $n = 200$ problems per length (yielding 436/600 [72.7%] pooled on seen lengths vs. 417/600 [69.5%] in Table 8B, and 85.0% vs. 86.0% at 15d, reflecting expected sampling variance within 95% Wilson confidence intervals). The accuracy cliff tracks the theoretical coordinate support boundary $p_{\max} = N_{\text{train}} + k$: $k = 0$ collapses at 16–25d ($p_{\max} = 15$), $k = 16$ at 35–40d ($p_{\max} = 31$), $k = 32$ at 45–50d ($p_{\max} = 47$), and $k = 64$ sustains out to 55d ($p_{\max} = 79$). The p-only ablation ($\tau = 0$) achieves 99.8% in-distribution and 96.2% across 16–30d, demonstrating that the macro column coordinate p carries the primary length-invariant scaffold, while τ provides boundary stabilization (58.0% vs. 41.5% at 40d). Retraining 1D-RoPE for 6,000 steps with tuned learning rate ($\text{LR} = 8 \times 10^{-4}$) yields only 28.0% at 8d and 0.0% beyond, confirming an in-distribution optimization bottleneck under flat coordinates in 1024-token contexts.

Evaluation Digit Length	2D-RoPE ($k = 0$)	2D-RoPE ($k = 16$)	2D-RoPE ($k = 32$)	2D-RoPE ($k = 64$)	p-Only ($\tau = 0, k = 32$)	1D-RoPE Retrained
Coordinate Horizon	$p_{\max} = 15$	$p_{\max} = 31$	$p_{\max} = 47$	$p_{\max} = 79$	$p_{\max} = 47$	6k steps, $\text{LR} = 8 \times 10^{-4}$
Seen 8 Digits ($n = 200$)	100.0% (200/200)	100.0% (200/200)	60.0% (120/200)	99.5% (199/200)	99.5% (199/200)	28.0% (56/200)
Seen 12 Digits ($n = 200$)	100.0% (200/200)	99.5% (199/200)	73.0% (146/200)	99.0% (198/200)	100.0% (200/200)	0.0% (0/200)
Seen 15 Digits (Max Seen, $n = 200$)	100.0% (200/200)	100.0% (200/200)	85.0% (170/200)	99.5% (199/200)	100.0% (200/200)	0.0% (0/200)
Unseen 16 Digits ($n = 200$)	77.5% (155/200)	100.0% (200/200)	86.0% (172/200)	100.0% (200/200)	100.0% (200/200)	0.0% (0/200)
Unseen 20 Digits ($n = 200$)	17.0% (34/200)	100.0% (200/200)	85.0% (170/200)	98.0% (196/200)	99.5% (199/200)	0.0% (0/200)
Unseen 25 Digits ($n = 200$)	0.0% (0/200)	99.5% (199/200)	89.0% (178/200)	93.5% (187/200)	99.0% (198/200)	0.0% (0/200)
Unseen 30 Digits ($n = 200$)	0.0% (0/200)	98.5% (197/200)	85.5% (171/200)	88.5% (177/200)	97.5% (195/200)	0.0% (0/200)
Unseen 35 Digits ($n = 200$)	0.0% (0/200)	**60.5% (121/200)**	75.5% (151/200)	83.0% (166/200)	85.0% (170/200)	0.0% (0/200)
Unseen 40 Digits ($n = 200$)	0.0% (0/200)	0.0% (0/200)	58.0% (116/200)	69.0% (138/200)	41.5% (83/200)	0.0% (0/200)
Unseen 45 Digits ($n = 200$)	0.0% (0/200)	0.0% (0/200)	**30.5% (61/200)**	60.0% (120/200)	9.5% (19/200)	0.0% (0/200)
Unseen 50 Digits ($n = 200$)	0.0% (0/200)	0.0% (0/200)	0.0% (0/200)	41.0% (82/200)	0.0% (0/200)	0.0% (0/200)
Unseen 55 Digits ($n = 200$)	0.0% (0/200)	0.0% (0/200)	0.0% (0/200)	**20.0% (40/200)**	0.0% (0/200)	0.0% (0/200)
Unseen 60 Digits ($n = 200$)	0.0% (0/200)	0.0% (0/200)	0.0% (0/200)	1.0% (2/200)	0.0% (0/200)	0.0% (0/200)
Unseen 70–80 Digits ($n = 400$)	0.0% (0/400)	0.0% (0/400)	0.0% (0/400)	0.0% (0/400)	0.0% (0/400)	0.0% (0/400)
Observed Cliff Location	$d = 16$–25	$d = 31$–40	$d = 45$–50	$d = 55$–65	$d = 40$–50	**No learning ($d \geq 12$)**
Theoretical Horizon ($p_{\max}$)	15	31	47	79	47	N/A

of the training datasets.

3.1 Atomic Character and Structural Control Tokenizer ($V = 115$)

To eliminate BPE subword chunking distortions while retaining clear structural boundaries for multi-turn agen-

Table 9: Exact Layer-by-Layer Parameter Topology of CBT-18.95M
$d_{\text{model}} = 512$, $n_{\text{head}} = 8$ ($d_k = 64$), $L = 6$ physical blocks, $R = 3$ recurrent passes ($L_{\text{eff}} = 18$), bias disabled, tied input/output embeddings.

Component / Module	Tensor Shape	Parameters	% of Total
Token Embedding (W_{te})	115×512	58,880	0.31%
Per Physical Block ($l \in \{1..6\}$):			
Pre-Attn LayerNorm (`ln_1`)	2×512	1,024	0.005%
Fused QKV Proj (`c_attn`)	512×1536	786,432	4.15%
Attention Output Proj (`c_proj`)	512×512	262,144	1.38%
Pre-MLP LayerNorm (`ln_2`)	2×512	1,024	0.005%
MLP Up-Proj (`c_fc`, 4×)	512×2048	1,048,576	5.54%
MLP Down-Proj (`mlp.c_proj`)	2048×512	1,048,576	5.54%
Subtotal per Physical Block	—	*3,147,776*	*16.62%*
All 6 Physical Blocks ($L = 6$)	$6 \times 3,147,776$	18,886,656	99.69%
Recurrent Unrolling ($R = 3$)	Weight-Tied ($L_{\text{eff}} = 18$)	+0	0.00%
Decoupled 2D-RoPE Cache	Non-Parametric (cos, sin)	+0	0.00%
Final LayerNorm (`ln_f`)	2×512	1,024	0.005%
LM Output Head ($W_{\text{lm_head}}$)	Tied to W_{te}	+0	0.00%
Non-Embedding Parameters	—	**18,887,680**	**99.69%**
Total Model Parameters	—	**18,946,560**	**100.00%**

tic dialogues, our tokenizer (`ScaledTokenizer`) defines a compact vocabulary $\mathcal{V}$ of size $V = 115$ for CBT-18.95M (the smaller CBT-200k and 100k variants use character subsets with $V = 111$ and $V = 27$ respectively), partitioned into two disjoint subsets:

$$\mathcal{V} = \mathcal{V}_{\text{ctrl}} \cup \mathcal{V}_{\text{char}}, \quad |\mathcal{V}_{\text{ctrl}}| = 14, \quad |\mathcal{V}_{\text{char}}| = 101 \tag{3}$$

1. **Structural Control Tokens ($\mathcal{V}_{\text{ctrl}}$, IDs $0 \dots 13$)**: Consists of 4 standard sequence control tokens (<PAD>, <BOS>, <EOS>, <UNK>) and 10 paired discourse state delimiters that structure agentic reasoning: <USER>, </USER>, <THINK>, </THINK>, <CALL>, </CALL>, <RESULT>, </RESULT>, <ANSWER>, </ANSWER>.
2. **Atomic ASCII Characters ($\mathcal{V}_{\text{char}}$, IDs $14 \dots 114$)**: Contains all printable ASCII characters—lowercase letters (a{z), uppercase letters (A{Z), single digits (0{9), punctuation symbols (+, =, (,), {, }, ", etc.), space, newline, and tab.

During tokenization, a greedy left-to-right prefix scanner checks if the input substring at index i matches any tag in $\mathcal{V}_{\text{ctrl}}$. If matched, the entire tag (e.g., <THINK>) is emitted as a single token ID; otherwise, the single character $s[i]$ is emitted as its atomic character ID.

3.2 Decoupled 2D Rotary Position Embeddings (2D-RoPE)

In **Decoupled 2D-RoPE**, every token x_t is associated with a 2D coordinate pair $(p_t, \tau_t) \in \mathbb{N}^2$, where p_t is the **Macro (Row) Coordinate** and τ_t is the **Micro (Column) Coordinate**. Within each attention head of dimension $d_k = 64$, we split the query and key vectors into two orthogonal 32-dimensional subspaces:

$$\tilde{\mathbf{q}}_t^{(h)} = \begin{bmatrix} \mathcal{R}_\Theta(p_t)\, \mathbf{q}_{t,:32}^{(h)} \\ \mathcal{R}_\Theta(\tau_t)\, \mathbf{q}_{t,32:}^{(h)} \end{bmatrix}, \quad \tilde{\mathbf{k}}_t^{(h)} = \begin{bmatrix} \mathcal{R}_\Theta(p_t)\, \mathbf{k}_{t,:32}^{(h)} \\ \mathcal{R}_\Theta(\tau_t)\, \mathbf{k}_{t,32:}^{(h)} \end{bmatrix} \tag{4}$$

For any coordinate scalar $c \in \{p_t, \tau_t\}$ and a 32-dimensional subspace vector $\mathbf{u} \in \mathbb{R}^{32}$, $\mathcal{R}_\Theta(c)\mathbf{u}$ applies 2×2 planar rotations across the 16 channel pairs $(2j, 2j+1)$ for $j \in \{0, 1, \dots, 15\}$ with $\theta_j = 10000^{-2j/32}$.

Proposition 1 (Zero Phase Drift & Scale-Invariant Column Alignment). *Let q_t be a scratchpad query token in decimal column p and k_j be the corresponding operand digit in column p. Under Decoupled 2D-RoPE (Eq. 4), the pre-softmax attention logit $\langle \tilde{\mathbf{q}}_t^{(h)}, \tilde{\mathbf{k}}_j^{(h)} \rangle / \sqrt{d_k}$ is strictly independent of the total number of digits N:*

$$\left(\tilde{\mathbf{q}}_t^{(h)}\right)^\top \tilde{\mathbf{k}}_j^{(h)} = \left(\mathbf{q}_{t,:32}^{(h)}\right)^\top \mathbf{k}_{j,:32}^{(h)} + \left(\mathbf{q}_{t,32:}^{(h)}\right)^\top \mathcal{R}_\Theta(\tau_t)\, \mathbf{k}_{j,32:}^{(h)} \tag{5}$$

Proof. Because both the scratchpad token and the prompt operand digit belong to decimal column 10^p, $p_t = p_j = p \implies \Delta p = 0 \implies \mathcal{R}_\Theta(0) = \mathbf{I}_{32}$, while $\tau_t \in \{0, \dots, 9\}$ is strictly bounded for all $N \in \mathbb{N}$.

The proposition is conditional on the coordinates being assigned correctly: it states that a single logit between a scratchpad query token and its corresponding operand digit does not depend on N, not that the model computes carries correctly or that the coordinates are produced by the model. Moreover, Proposition 1 covers only scratchpad-to-operand attention; for the final answer digits after </THINK>=, the parser sets $p = \max(0, C - 1 - i)$ using the model's emitted scratchpad column count C, so answer-to-scratchpad alignment depends on the model having emitted the right number of

scratchpad columns. Since RoPE depends only on relative offsets, the reason shift masking is considered during training is not absolute position but the large offset between scratchpad rows ($p > 0$) and the prompt tokens at $p = 0$ that appears at test length N but was never seen at training length.

3.3 Coordinate Assignment at Inference

The coordinates are computed by a deterministic, hand-written rule-based parser (`compute_2d_coords` in `champion_model/scaled_t4/model.py`) that is called inside the forward pass over the token ids; it is applied identically in training and in autoregressive generation, and the model does not predict (p, τ). The rules are: - *Operands*: a digit in the first (second) operand gets p equal to its distance from the `+` (`=`) token minus one, so units digits have $p = 0$; $\tau = 0$. - *Scratchpad*: inside <THINK>, each `(` token starts a new column (p increments, τ resets to 0), and τ counts tokens since the `(`, capped at 9. The model therefore decides when to advance a column by emitting `(`; the parser does not count 10-character templates. Because column advancement is keyed on `(`, the model can advance p only by emitting `(`, which also means a single spurious or missing `(` would shift all subsequent column coordinates. Across all 240 random 16–50 digit problems (Table 3B), the 20 structured 16–50 digit problems, and the 14 ablation cases (Models 3–5 and `CBT-100k Ref`), **zero errors arose from a mis-emitted `(` inside** <THINK>; `(` column-count errors appeared only at $d \geq 60$ once macro coordinates exceeded the shift-masked training horizon $p_{\max} = 47$ (3/20 cases at 60d, 4/10 at 80d, 1/10 at 100d). The 1D baselines, by contrast, failed by repeating valid 10-character `(0+0+0)=00` blocks too many times rather than corrupting the `(` delimiter syntax. - *Answer*: after </THINK>=, the i-th digit emitted gets $p = \max(0, C - 1 - i)$ where C is the number of scratchpad columns. The answer length is not given to the model; extra digits are clamped at $p = 0$, so the unit digit ($i = C - 1$) and any extra overflow digit ($i \geq C$) receive the same coordinate $p = 0$. The model must therefore identify the termination point itself, and this $p = 0$ clamp—alongside the $p = 1 \to p = 0$ shift-masking boundary—is a plausible cause of the off-by-one readout error in Table 8. - *Discourse mode*: if <USER> is present, $p \in \{0, \ldots, 4\}$ is the index of the most recent segment tag (<USER>, <THINK>, <CALL>, <RESULT>, <ANSWER>) and τ counts tokens since that tag.

Thus the operand-to-column alignment (the operand and column ranks) is supplied by the parser, and the length-generalization result should be read as conditional on this hand-designed alignment. The parser also assumes a fixed prompt format (`A+B=`), and is a Python loop over the batch that is not optimized for speed.

During training, we apply **Selective Coordinate Shift Masking**:

$$p_t \leftarrow p_t + \Delta \cdot \mathbb{I}(p_t > 0), \quad \Delta \sim \mathcal{U}\{0, 1, \ldots, k\} \tag{6}$$

with $k = 45$ for CBT-18.95M and $k \in \{0, 16, 32, 64\}$ for the 100k ablation models of Table 8. This exposes $p > 0$ arithmetic columns to indices up to $p = 70$ during 1–25-digit replay (with $k = 45$), while keeping the <USER> prompt ($p = 0$) anchored at the origin. It is closely related to randomized positional encodings [18].

3.4 Weight-Tied Recurrent Depth with $1/\sqrt{R}$ Scaling

We unroll the $L = 6$ physical Transformer blocks across $R = 3$ recurrent passes per forward step ($L_{\text{eff}} = 18$ layers at zero parameter overhead) using a $1/\sqrt{R}$-scaled residual update:

$$\mathbf{x} \leftarrow \mathbf{x} + \frac{1}{\sqrt{R}} \left(\text{Block}_l(\mathbf{x}, p, \tau) - \mathbf{x} \right) \tag{7}$$

The $1/\sqrt{R}$ factor is a heuristic residual damping choice. We do not derive a variance bound for it, and our ablation (Table 8) shows no benefit from recurrence for addition. We therefore drop the variance comparison between $1/\sqrt{R}$ scaling and unscaled accumulation that appeared in earlier drafts.

3.5 Three-Stage Training Data & 1-Hop Induction Curriculum

Table 10 summarizes the three-stage protocol that produced the pre-SFT base `champion_base.pt` on dual NVIDIA T4 GPUs (FP16 AMP). The champion checkpoint (`champion_final.pt`) starts from that base and is trained further with the V4 SFT curriculum (`sft_search.py --curriculum v4`: 45% multi-tool, 35% search copy, 20% arithmetic replay with 1–25-digit operands, $k = 45$; hyperparameters in the training script, not in Table 10): 1. **Stage 1 (Joint Pretraining, 45,059 steps, 247.3M tokens, Loss `0.000094`)**: Trained on a 50%/50% mixture of 1–15 digit column addition (40% asymmetric lengths, 25% symmetric lengths, 20% cascading carries $99\ldots9 + b$, 15% power-of-ten zero chains) and 6-category structured tool dialogues, with -100 loss masking on prompts and <RESULT> payloads. 2. **Stage 2 (Search Copy Induction SFT, 7,000 steps, Loss `0.004834`)**: In early SFT tests, repeating an unfamiliar entity X inside <THINK> before calling <CALL> caused a *2-Hop Copy Bottleneck*, where character n-gram priors hallucinated phonotactic nonces (`"tomatoes"` → `"smoom"`, `"bloumplik"` → `"cheat"`). Decoupling <THINK> into an invariant intent statement and training on 65% procedurally generated phonotactic pseudowords

Table 10: Complete Three-Stage Training Data Composition, Digit Horizon Boundaries, and Loss Masking Specification for the CBT-18.95M Pre-SFT Base (`champion_base.pt`; the champion adds V4 SFT)
Across all stages, arithmetic training is strictly bounded to **1–15 digits** ($N \leq 15$). All 16- to 50-digit evaluations are 100% zero-shot out-of-distribution extrapolations.

Training Stage	Dataset Component / Sub-Distribution	Pattern / Schema Specification	Trained Digit / Length Scope	Mixture Weight	Supervised Target (Masking Rule)
Stage 1: Pretraining	**A. Arithmetic State-Machine Corpus (50%)**	$A + B =$ <THINK>$(c+a+b) = c's \ldots$ </THINK>$= S$	**Strictly 1–15 Digits** ($N \leq 15$)	**50.0% of Pretrain**	Prompt $A + B =$ masked (-100)
	1. Asymmetric & Symmetric Random Sums	$d_A, d_B \sim \mathcal{U}[1, 15]$ (40% asymmetric, 25% symmetric)	$1 \leq d_A, d_B \leq 15$ (0% $>$ 15d)	32.5% (65% of Math)	Scratchpad + Final Sum supervised
	2. Cascading Carry & Zero Chains	$99 \ldots 9 + b$ (20%) and $k_1 10^{d_1 - 1} + k_2 10^{d_2 - 1}$ (15%)	$2 \leq d \leq 15$ (0% $>$ 15d)	17.5% (35% of Math)	Scratchpad + Final Sum supervised
	B. Multi-Domain Tool Corpus (50%)	<USER> $\rightarrow$ <THINK> $\rightarrow$ <CALL> $\rightarrow$ <RESULT> $\rightarrow$ <ANSWER>	$T \leq 512$ Characters	**50.0% of Pretrain**	<USER> & <RESULT> masked (-100)
	Single-Tool, Chaining, JSON, Error, Chat	6 interaction categories across cities, timezones, units	1–3 Conversational Turns	50.0% (6 Equal Cats)	<THINK>, <CALL>, <ANSWER>
Stage 2: Search SFT	**Open-Domain Search 1-Hop Copy (7k steps)**	Entity-agnostic <THINK> + `search(query="X")`	65% Dynamic Nonces / 35% Real	**81.25% (26/32 Batch)**	Forces $L_{\text{memorize}} \to \infty$ pointer copy
	Math & Multi-Tool Replay	1–15d carry chains + tool schemas	**Strictly 1–15 Digits** ($N \leq 15$)	**18.75% (6/32 Batch)**	Preserves 50d math & tool syntax
Stage 3: All-Tools SFT	**Universal 9-Tool 1-Hop Annealing (4k steps)**	All 9 tools (`weather, time, unit, calc, db, api, search...`)	Varied Prepositions & Nonces	**90.0% of Stage 3**	Resolves preposition/routing drift
	High-Digit Arithmetic Carry Replay	Cascading carries & random sums ($N \leq 15$d)	**Strictly 1–15 Digits** ($N \leq 15$)	**10.0% of Stage 3**	Maintains 100.0% 1–50d math

($L_{\text{memorize}} \to \infty$) forced a **1-hop induction pointer head** ($p = 2 \to p = 0$) to copy unseen strings character-by-character. 3. **Stage 3 (All-Tools 1-Hop Annealing SFT, 4,000 steps, LR** $5 \times 10^{-5} \to 5 \times 10^{-6}$**, Loss** `0.007532`**)**: Extended the 1-hop pointer copy formulation across all 9 tool schemas (`get_weather`, `get_time`, `calculator`, `unit_convert`, `database_query`, `fetch_api`, `create_event`, `file_manager`, and `search`) alongside a 10% 1–15 digit math replay buffer, achieving **100.0% (26/26) tool routing**, **96.2% (25/26) exact argument extraction**, and **100.0% 1–50 digit sum extrapolation**.

3.6 Hyperparameters and Release

Defaults in the training code (`train_and_eval.py` and `sft_search.py` under `champion_model/scaled_t4/`): AdamW with weight decay 0.01, peak learning rate 10^{-3} with cosine decay, gradient-norm clipping at 1.0, per-GPU batch size 32 on 2 GPUs, context length (block size) 1024 tokens with up to 4096 coordinate indices, $k = 32$ shift range, $R = 3$, FP16 autocast with gradient scaling. Stage 3 SFT uses a learning rate from 5×10^{-5} to 5×10^{-6}. At ≈ 14.3 characters per digit, a 50-digit trace requires ≈ 715 tokens (fitting within CBT's 1024 context window, but exceeding the 512-token window of the 12-digit baselines). Traces for 25 digits (≈ 360 tokens) and 35 digits (≈ 500 tokens) fit within 512 tokens, where the 12-digit baselines fail from positional drift rather than context clipping. At 70 digits (≈ 1000 tokens) and 80 digits (≈ 1149 tokens), traces approach or exceed 1024 tokens, confounding arithmetic accuracy with context truncation. The code, data generators, and evaluation scripts are available at github.com/NeithanT/CBT, and model checkpoints are hosted at huggingface.co/Neithanvarvas/CBT (primary checkpoint `champion_final.pt` SHA256: `79d908d3...ccaa`; 18.91M 1D-Char `model_1_final.pt` SHA256: `06411d79...0c1f`; 1.56M HRM-RoPE `hrm_rope_1000_examples.pt` SHA256: `18515686...f6ee`).

4. Discussion, Limitations, and Future Horizons

4.1 Positioning Within Prior Length-Generalization Work

Prior studies have shown that standard Transformers fail to extrapolate arithmetic beyond training lengths under 1D positional encodings [3, 5, 6, 8]. Several lines of work address this by modifying positional information or data formatting: Shen et al. [4], Zhou et al. [19], and Duan & Shi [20] use explicit index hints and structured scratchpads; Ruoss et al. [18] randomize positional indices across a wider range during training (closely related to our shift masking in Eq. 6); McLeish et al. [7] introduce Abacus embeddings that align digits of equal place value; and Fan et al. [21] use looped Transformers for iterative algorithms. Our Decoupled 2D-RoPE is philosophically similar to Abacus embeddings and explicit index hints in that it supplies place-value alignment (p, τ) via a rule-based parser rather than learning it from raw text, while extending the coordinate grid to a two-dimensional (p, τ) RoPE subspace that accommodates both multi-character column scratchpads and multi-segment tool dialogues.

4.2 Why CBT Suits Local Edge Inference Rather Than Brute-Force Scaling

A central takeaway of our work is that **CBT is not meant to replace subword tokenization in massive trillion-parameter cloud LLMs as-is**. Because CBT generates text *character-by-character* and executes $R = 3$ recurrent passes per character, generating a 50-word paragraph requires ~ 300 sequential forward passes (compared to ~ 65 subword passes in a BPE model). At billion-parameter scale, this character-by-character serial bottleneck would make inference prohibitively slow.

However, at compact scales (**100k to 18.95M parameters**), individual forward passes involve a small matrix footprint (36 MB of FP16 weights, though $R = 3$ recurrence triples the per-token FLOPs to roughly that of a ˜57M-parameter dense model and character-level sequences expand the KV cache). On an Intel Core Ultra 5 125H laptop CPU (FP32), batched KV-cached decoding runs at **141–394 character tokens/sec** across 16–100 digit traces (and 1.1–5.3 tokens/sec uncached single-stream); we did not benchmark mobile ARM hardware or INT8 quantization, nor did we compare against fine-tuned small off-the-shelf subword models on tool calling. Subject to those practical constraints, CBT's ability to handle intent classification, 9-schema tool routing (26/26 on template-matched prompts; 8/16 exact argument copying on OOD prompts), and 16–50 digit sums (237/240 random; 20/20 structured) within a 36 MB weight file suggests potential utility for: - **Low-Resource Local Tool Routers**: On-device assistants on mobile phones, embedded IoT controllers, or local desktop daemons that parse user commands into structured API/JSON calls (`get_weather`, `file_manager`, `database_query`) without cloud connectivity. - **Deterministic Symbolic Co-Processors**: Compact local modules that execute exact character-level formatting, arithmetic, and pointer extraction without subword hallucination.

4.3 Open Questions and Further Testing

While CBT-18.95M shows that placing characters on a hand-assigned 2D coordinate grid (p, τ) yields 100.0% accuracy (180/180) on random 16–40 digit sums, 100.0% (240/240) across 16–50 digits, and 100.0% (20/20) at 55 digits (70.0% at 60 digits) before degrading at 60 digits, with failures already appearing before the $p_{\max} = N_{\text{train}} + k = 70$ shift-masking horizon (Table 3B), our OOD and ablation experiments leave several questions open: 1. **Sensitivity to Epoch Reinforcement and Boundary Coordinates**: As shown in our 100k component ablations (Table 8), while several combinations of 2D-RoPE and recurrence achieved identical 78.6% strict match at 12.8 epochs, the off-by-one trailing zero readout error at the $p = 1 \rightarrow p = 0$ boundary disappeared in our runs with higher epoch density (64 epochs on a 1,000-sample pool) or larger capacity (18.95M); however, these single-run comparisons do not isolate whether the fix came from boundary exposure, memorization of the small pool, or model scale. 2. **Multi-Word OOD Pointer Spans and General Algorithms**: In zero-shot OOD testing (Table 5), two-word capitalized proper nouns (`"Buenos Aires"` $\rightarrow$ `"Buris"`) still experience partial compression, and our algorithmic generalization scope here focused on $\mathcal{O}(N)$ column sums rather than $\mathcal{O}(N^2)$ multiplication or arbitrary graph algorithms. 3. **Learned Coordinates and Other Tasks**: Emitting hundreds of scratchpad characters is an inefficient simulation of an ALU, the coordinates are hand-assigned, and extrapolation is bounded by the shift-masked coordinate support $p_{\max} = N_{\text{train}} + k$ (Table 3B). Important next steps are learning or inferring (p, τ), testing on a second task whose coordinates are not trivial (multiplication, sorting, string reversal), running multi-seed training, and comparing against equalized 1D baselines with randomized positions, index hints, and tuned Abacus embeddings [18, 19, 20, 21].

5. Conclusion

Starting from the failure of a standard 18.91M-parameter Transformer and eight architectural baselines on unseen 18- to 20-digit sums (**0.0% extrapolation at 18–20 digits**, despite 4.6–18.7% on moderate 14–16 digit sums), our experiments suggest that position geometry is the dominant factor (character-level 1D-

RoPE: 2.3% across 14–20d; character-level 2D: 100%), with subword tokenization secondary. We studied the **Character-Based Transformer (CBT)**, which generates atomic characters and structural control tags on a two-dimensional coordinate matrix (p, τ) via **Decoupled 2D-RoPE** and weight-tied recurrent depth ($R = 3$, which our ablation found unnecessary for addition). Trained strictly on 1- to 15-digit sums with $k = 32$ shift masking ($p_{\max} = 47$) and multi-tool dialogues, **CBT-18.95M** (`champion_final.pt`) solves 20/20 structured 16–50 digit sums, achieves **180/180 (100.0%)** on random multi-carry sums across 16–40 digits and **237/240 (98.8%)** across 16–50 digits (degrading beyond $p_{\max} = 47$ from 90.0% at 55d to 45.0% at 60d and 0.0% at 80–100d), routes 26/26 tool cases (25/26 exact arguments, on templates shared with training), and routes 16/18 OOD prompts, of which 8/16 (50.0%) have exact arguments. Recurrent depth gave no measurable benefit for addition in our 100k ablation. Ultimately, our findings show that while 2D embeddings and character tokenization achieve high performance with exceptional **memory efficiency**, character-by-character recurrence is **not compute-efficient** at scale—meaning current subword LLMs remain well-suited for general cloud AI, whereas CBT is a candidate design for memory-constrained local tool-calling agents and a starting point for studying coordinate-aware architectures, pending the broader tests listed above.

Disclaimer: Use of AI and Compute Infrastructure

- **Use of AI Assistance**: Portions of the experimental engineering, code implementation, data pipeline structuring, and manuscript drafting/formatting in this research were conducted with the assistance of **Google Gemini** via the Antigravity agentic coding environment. All experimental hypotheses, architectural selections, empirical benchmarks, and final scientific conclusions were verified and validated by the author.
- **Compute Hardware**: All model pretraining, supervised fine-tuning (SFT), and isolated component ablation runs reported in this paper were executed on **Kaggle's NVIDIA Tesla T4 GPUs** (16 GB GDDR6 VRAM, Turing `sm_75` architecture) using PyTorch native FP16 automatic mixed precision (`torch.float16`).

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[3] R. Nogueira, Z. Jiang, and J. Lin. Investigating the limitations of transformers with simple arithmetic tasks. arXiv preprint arXiv:2102.13019, 2021.

[4] R. Shen, S. Bubeck, R. Eldan, Y. T. Lee, Y. Li, and Y. Zhang. Positional description matters for transformers arithmetic. arXiv preprint arXiv:2311.14737, 2023.

[5] S. Jelassi, S. d'Ascoli, C. Domingo-Enrich, Y. Wu, Y. Li, and F. Charton. Length generalization in arithmetic transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[6] N. Dziri, X. Lu, M. Sclar, X. L. Li, L. Jiang, B. Y. Lin, S. Welleck, P. West, C. Bhagavatula, R. Le Bras, et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[7] S. McLeish, A. Bansal, A. Stein, N. Jain, J. Kirchenbauer, B. R. Bartoldson, B. Kailkhura, A. Bhatele, J. Geiping, A. Schwarzschild, and T. Goldstein. Transformers can do arithmetic with the right embeddings. In *International Conference on Machine Learning (ICML)*, 2024.

[8] A. Kazemnejad, I. Padhi, K. Natesan Ramamurthy, P. Das, and S. Reddy. The impact of positional encoding on length generalization in transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[9] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[10] M. Nye, A. J. Andreassen, G. Gur-Ari, H. Michalewski, J. Austin, D. Bieber, D. Dohan, A. Lewkowycz, M. Bosma, D. Luan, et al. Show your work: Scratchpads for intermediate computation with language models. arXiv preprint arXiv:2112.00114, 2021.

[11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing*

Systems (NeurIPS), volume 35, pages 24824–24837, 2022.

[12] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.

[13] A. Giannou, S. Rajput, J.-y. Sohn, K. Lee, J. D. Lee, and D. Papailiopoulos. Looped transformers as programmable computers. In *International Conference on Machine Learning (ICML)*, pages 11398–11442, 2023.

[14] R. Al-Rfou, D. Choe, N. Constant, M. Guo, and L. Jones. Character-level language modeling with deeper self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.

[15] L. Xue, A. Barua, N. Constant, R. Al-Rfou, S. Narang, M. Kale, A. Roberts, and C. Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022. arXiv:2105.13626.

[16] A. Pagnoni, R. Pasunuru, P. Rodriguez, J. Nguyen, B. Muller, M. Li, C. Zhou, L. Yu, J. Weston, L. Zettlemoyer, G. Ghosh, M. Lewis, A. Holtzman, and S. Iyer. Byte latent transformer: Patches scale better than tokens. arXiv preprint arXiv:2412.09871, 2024.

[17] Y. Shaham and O. Levy. Neural machine translation without embeddings. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2021.

[18] A. Ruoss, G. Deletang, T. Genewein, J. Grau-Moya, R. Csordas, M. Bennani, S. Legg, and J. Veness. Randomized positional encodings boost length generalization of transformers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[19] Y. Zhou, U. Alon, X. Chen, X. Wang, R. Agarwal, and D. Zhou. Transformers can achieve length generalization but not robustly. arXiv preprint arXiv:2402.09371, 2024.

[20] S. Duan and Y. Shi. From interpolation to extrapolation: Complete length generalization for arithmetic transformers. arXiv preprint arXiv:2310.11984, 2023.

[21] Y. Fan, Y. Du, K. Ramchandran, and K. Lee. Looped transformers for length generalization. arXiv preprint arXiv:2409.15647, 2024.